COLLEGE CLUB MANAGEMENT SYSTEM

Python-Based Terminal Application

Student Name: Eliza Khan
Registration No.: 26MIM10195

Prepared for VITyarthi Project Evaluation

Project at a Glance

Item	Project detail
Interface	Command-Line / Terminal
Language	Python 3
External packages	None
Storage	Local TXT files
Core modules	Clubs, Members, Events, Registrations

1. Introduction

The College Club Management System is a simple terminal-based Python application designed to organize college club information. It manages clubs, associates members with clubs, creates events, and records event registrations. The system uses Python modules and local text files, making it lightweight and easy to run from the command line.

2. Problem Statement

College clubs may manage member and event information manually, making it difficult to keep records organized and consistent. The proposed system provides a small digital solution for maintaining clubs, members, events, and registrations through a terminal interface.

3. Objectives

- Maintain each club with a unique club ID.
- Maintain member records linked to existing clubs.
- Create events linked to existing clubs.
- Register members for events and prevent duplicate registrations.
- Demonstrate Python functions, conditional logic, loops, file handling, validation, and modular programming.

4. Scope and Target Users

The scope covers club management, member management, event management, and event registration. Target users are students and college club coordinators. The current version does not include GUI, web services, cloud databases, authentication, notifications, payments, or external databases.

5. Functional Requirements

ID	Functional requirement	Module
FR-01	Add a club and prevent duplicate club IDs.	clubs.py
FR-02	View all stored clubs.	clubs.py
FR-03	Search clubs by ID or name.	clubs.py
FR-04	Add a member and link it to an existing club.	members.py
FR-05	View club members.	members.py
FR-06	Add an event linked to an existing club.	events.py
FR-07	View stored events.	events.py
FR-08	Register a member for an event and prevent duplicates.	registrations.py
FR-09	View event registrations.	registrations.py

6. Non-Functional Requirements

Requirement	Application in the project
Usability	Terminal menu provides direct prompts and simple text output.
Reliability	Duplicate club/member/event/registration checks reduce inconsistent records.
Maintainability	Separate modules isolate clubs, members, events, registrations, storage, and validation.
Error handling	Empty IDs/details, missing club/member/event references, invalid dates, and duplicate records are rejected wit
Resource efficiency	Uses local text files and standard Python only; no external service is required.
Portability	Can run from the command line on a system with Python 3 installed.

7. System Architecture

The application follows a modular command-line architecture. main.py controls navigation. Domain modules manage clubs, members, events, and registrations. database.py provides file read/write operations, while validators.py contains basic validation helpers. Four text files in the data directory act as persistent storage.

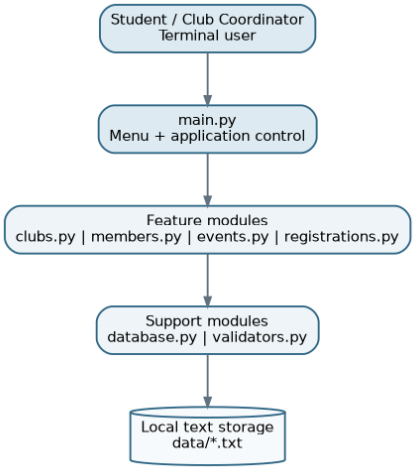


Figure 1. System architecture

8. Workflow Diagram

The main workflow begins with the menu, continues through the selected management operation, and reads or writes the corresponding local text file.

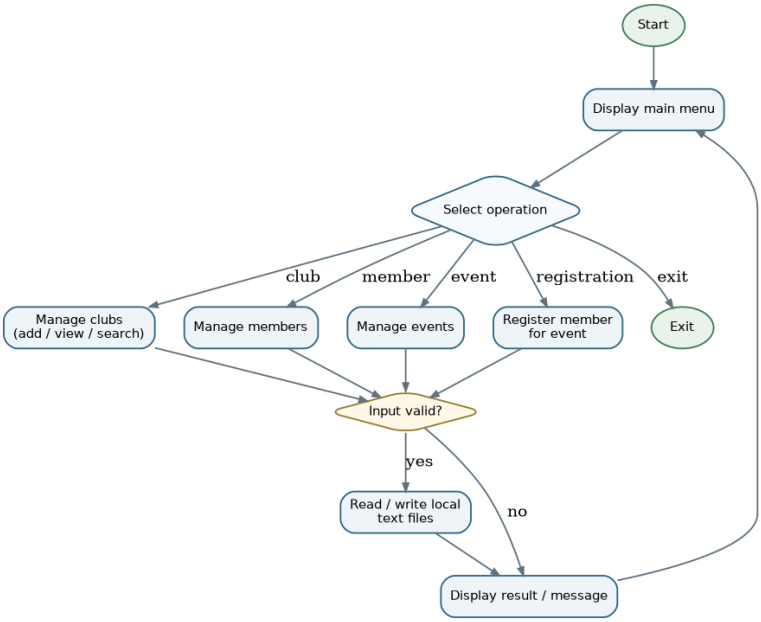


Figure 2. Main process workflow

9. Use Case Diagram

The primary actor is a Student / Club Coordinator. The actor can manage clubs, members, and events, search clubs, and register members for events through the terminal interface.

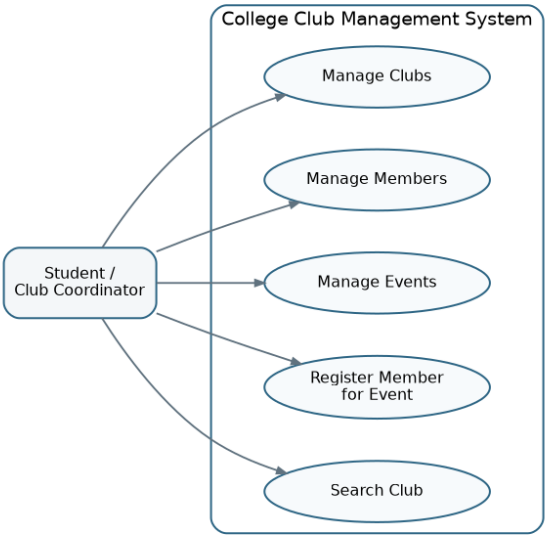


Figure 3. Use case diagram

10. Sequence Diagram

A representative interaction shows the user selecting an operation, main.py invoking a domain module, the module using database.py for local storage, and the result being returned to the user.

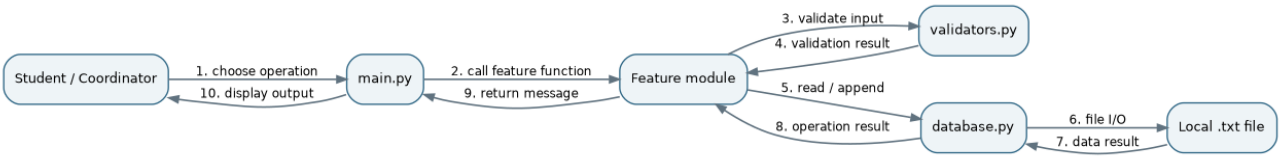


Figure 4. Representative interaction sequence

11. Class / Component Diagram

The implementation is module-oriented rather than class-oriented. main.py is the controller; clubs.py, members.py, events.py, and registrations.py provide feature operations; database.py provides persistence helpers; validators.py provides input checks.

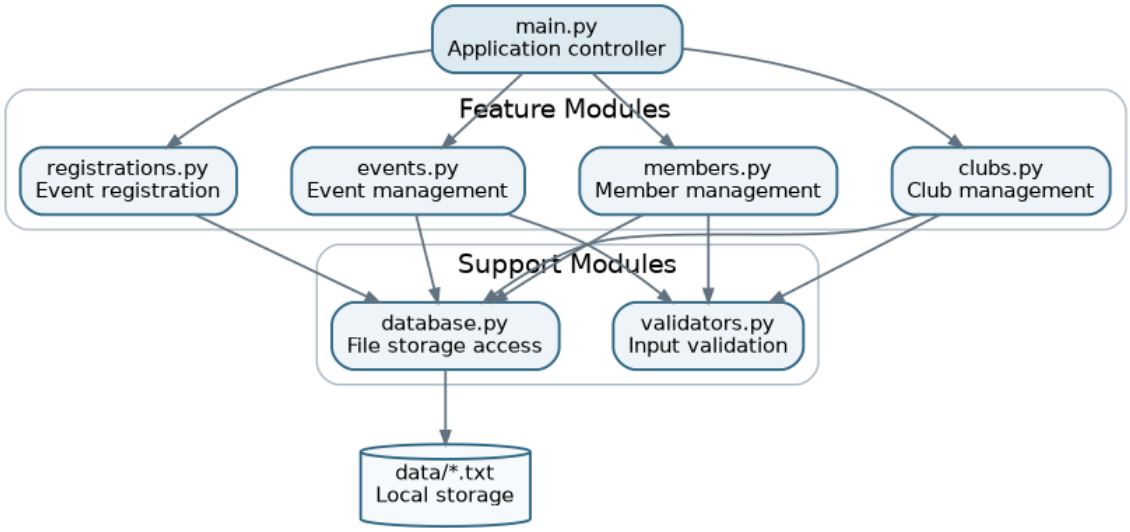


Figure 5. Module/component diagram

12. Storage Design

The project uses text-file storage rather than a relational database. Each record is pipe-delimited and stored in a dedicated file. The registration file references a member ID and an event ID, while member and event records reference a club ID.

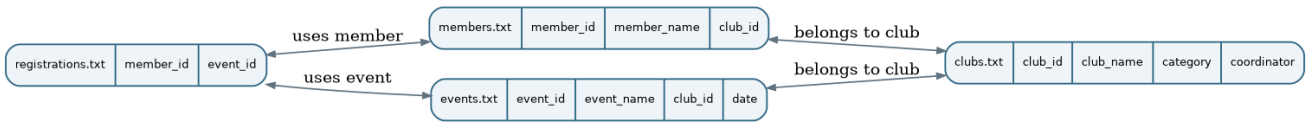


Figure 6. Storage/schema relationship view

13. Design Decisions & Rationale

Modular structure: Separate files for clubs, members, events, registrations, storage, and validation keep responsibilities understandable and maintainable.

Local TXT storage: Matches the educational scope and avoids external database dependencies while demonstrating file handling.

Relationship checks: A member must reference an existing club, an event must reference an existing club, and registration requires existing member/event IDs.

Duplicate prevention: Unique ID and registration checks reduce accidental duplicate records.

Terminal interface: Keeps the application focused on Python fundamentals and direct command-line interaction.

14. Implementation Details

File	Responsibility
main.py	Displays the menu, reads choices, and routes operations.
clubs.py	Adds, views, and searches club records.
members.py	Adds and views members and validates club references.
events.py	Adds and views events and validates club/date input.
registrations.py	Registers members for events and prevents duplicates.
database.py	Reads and appends local text-file records.
validators.py	Provides basic text, ID, and date-format validation.

15. Project Structure

The supplied project contains seven Python source modules plus README.md, statement.md, TESTING.md, requirements.txt, .gitignore, and four local data files. This satisfies the project’s modular organization requirement and provides a clear separation of responsibilities.

16. Screenshots / Results

The following screenshots were produced from actual executions of the supplied project. The scenarios demonstrate club creation, member/event creation, registration, search, duplicate prevention, and validation handling.

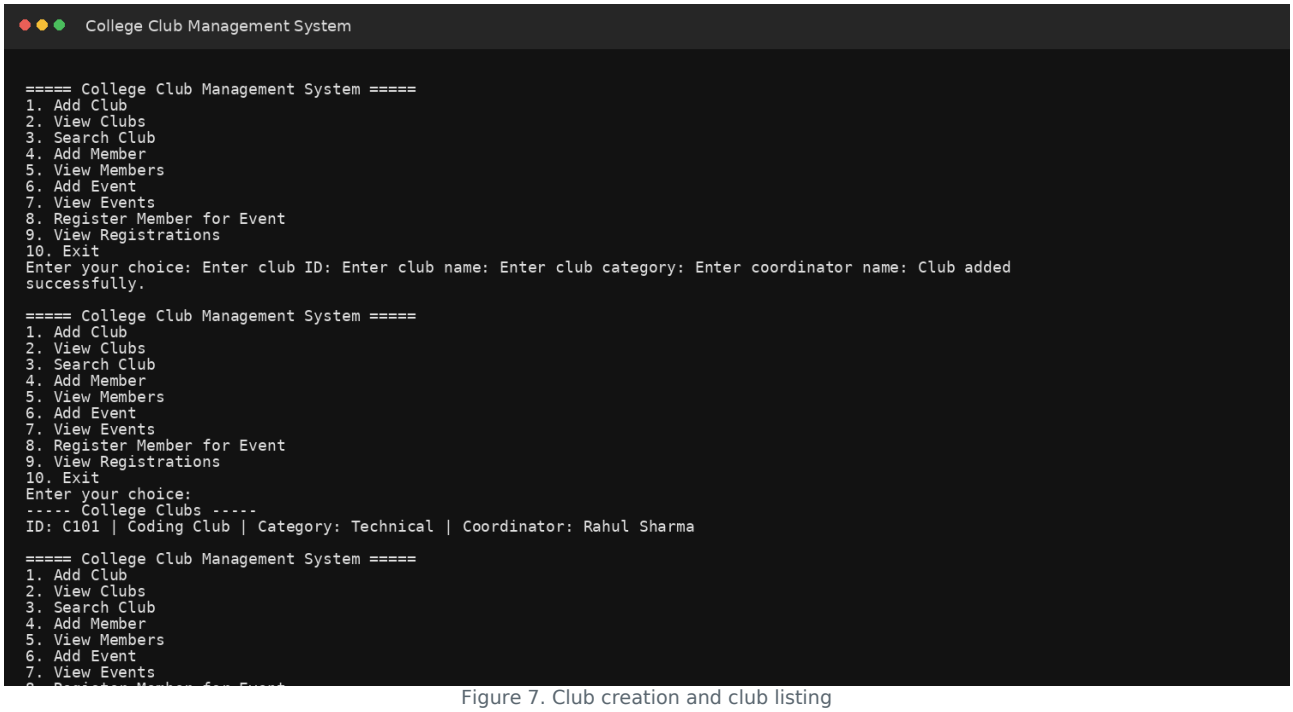


Figure 7. Club creation and club listing

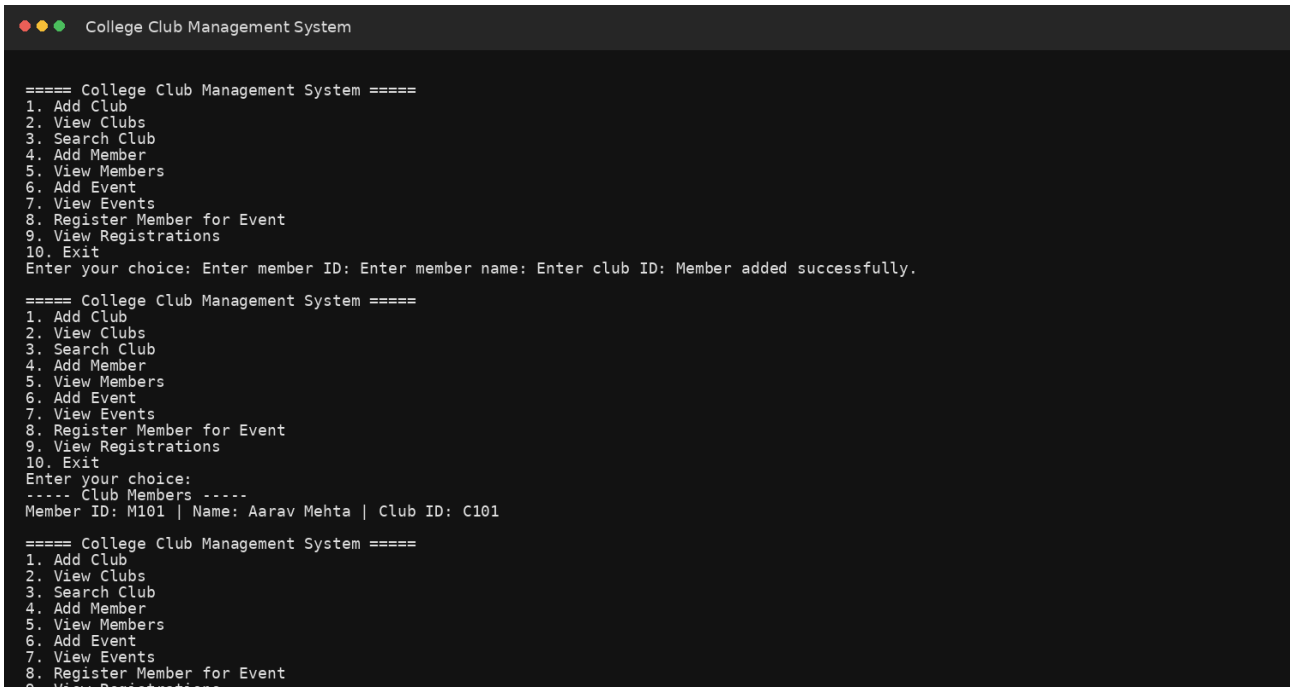


Figure 8. Member, event, and registration operations

16.1 Search and Validation Results

College Club Management System

```
===== College Club Management System =====
1. Add Club
2. View Clubs
3. Search Club
4. Add Member
5. View Members
6. Add Event
7. View Events
8. Register Member for Event
9. View Registrations
10. Exit
Enter your choice: Enter club ID or name to search: ID: C101 | Coding Club | Category: Technical | Coordinator:
Rahul Sharma

===== College Club Management System =====
1. Add Club
2. View Clubs
3. Search Club
4. Add Member
5. View Members
6. Add Event
7. View Events
8. Register Member for Event
9. View Registrations
10. Exit
Enter your choice: Enter club ID: Club already exists.

===== College Club Management System =====
1. Add Club
2. View Clubs
3. Search Club
4. Add Member
5. View Members
6. Add Event
7. View Events
8. Register Member for Event
9. View Registrations
```

Figure 9. Club search and duplicate-club handling

College Club Management System

```
===== College Club Management System =====
1. Add Club
2. View Clubs
3. Search Club
4. Add Member
5. View Members
6. Add Event
7. View Events
8. Register Member for Event
9. View Registrations
10. Exit
Enter your choice: Enter member ID: Enter member name: Enter club ID: Club ID not found.

===== College Club Management System =====
1. Add Club
2. View Clubs
3. Search Club
4. Add Member
5. View Members
6. Add Event
7. View Events
8. Register Member for Event
9. View Registrations
10. Exit
Enter your choice: Enter event ID: Enter event name: Enter club ID: Enter event date (YYYY-MM-DD): Invalid date
format.

===== College Club Management System =====
1. Add Club
2. View Clubs
3. Search Club
4. Add Member
5. View Members
6. Add Event
7. View Events
8. Register Member for Event
9. View Registrations
```

Figure 10. Invalid references and duplicate-registration handling

17. Testing Approach

Testing was performed through command-line scenarios based on the supplied TESTING.md. The checks cover normal operations, invalid references, duplicate records, invalid date format, search, and menu validation.

Test case	Expected / observed behavior
Add club	Club record stored and displayed successfully.
Duplicate club ID	Existing ID is rejected.
Search club	Matching club is displayed; missing search result reports no match.
Add member with existing club	Member record is stored successfully.
Add member with missing club	Operation is rejected.
Add event	Event record is stored when the club exists.
Invalid event date format	Operation is rejected.
Register member for event	Registration is stored successfully.
Duplicate registration	Existing registration is rejected.
Invalid menu choice	Clear invalid-choice message is shown.

18. Challenges Faced

- Maintaining consistent relationships between club IDs, member records, event records, and registrations.
- Preventing duplicate IDs and duplicate event registrations while keeping the implementation simple.
- Validating required fields and basic date format without external libraries or a database.
- Keeping the workflow modular while retaining a straightforward command-line experience.

19. Learnings & Key Takeaways

- Modular Python program design.
- Practical file handling for persistent data.
- Conditional validation and relationship checks.
- Managing related records using IDs.
- Designing and testing a complete command-line workflow.

20. Future Enhancements

The supplied README identifies several possible future improvements: club-admin login, event attendance, announcements and reminders, SQLite/MySQL support, a Tkinter GUI, and CSV/PDF reports. These are proposed enhancements and are not represented as implemented features in the current source.

21. Limitations

The current implementation does not provide a GUI, web server, cloud database, authentication, notifications, payments, or an external database. Date validation checks the YYYY-MM-DD format pattern but does not validate calendar correctness. The system is intended as a basic educational terminal application.

22. References

• VITyarthi — Build Your Own Project: General Project Instructions & Submission Guidelines. The supplied guideline requires functional/non-functional requirements, architecture and design artefacts, a modular project structure, testing, README.md, statement.md, and a detailed PDF report. filecite turn9file0 L109-L134
• Project README.md, statement.md, TESTING.md, and supplied Python source modules.

23. Author / Submission Details

Field	Details
Student Name	Eliza Khan
Registration No.	26MIM10195
Course	Python Essentials
Project	College Club Management System
Interface	Command-Line / Terminal
Storage	Local Text Files

End of Project Report